BACSE344 - Cloud Infrastructure and Architecture

Digital Assignment 3: Application Scaling and Security | C1 Slot | Academic Year 2026

Student Name:	Mugunth	**Application:**	Campus Resource Booking System
GitHub Branch:	github.com/Mugu1825/.../tree/assignment-3	**Domain:**	Institute / University Scheduling
Assignment 1 Live:	campus-resource-booking-cqt7.onrender.com	**Cloud Database:**	MongoDB Atlas Replica Set (M0)

QUESTION 1 — SCALE YOUR APPLICATION (5 MARKS)

1(a) Peak-Demand Situation & 3-Instance Backend behind Nginx Load Balancer (1 Mark)

Peak-Demand Scenario: In our university domain, peak demand occurs during the **'Semester Lab Slot & 48-Hour Hackathon Registration Rush'** at 9:00 AM on Monday. Over 1,500 students across 11 departments concurrently hit the booking system to secure computer laboratories (e.g., AI/ML Lab, IoT Lab), seminar halls, and project rooms before slots run out.
API Hit Hardest: GET /api/resources/<id>/availability?date=... and GET /api/resources. Each student continuously queries available 1-hour slots (09:00 to 21:00) and multi-day hackathon intervals before submitting.
Implementation: Deployed 3 backend instances (crbs-backend-1:5001, crbs-backend-2:5002, crbs-backend-3:5003) running Gunicorn behind an Nginx Load Balancer (crbs-nginx-lb:8080) configured with round-robin upstream balancing and connected to MongoDB Atlas.

Req #	Endpoint URL	Served by Backend Instance	Response Status
01	/api/instance	**backend-instance-1**	HTTP 200 OK
02	/api/instance	**backend-instance-2**	HTTP 200 OK
03	/api/instance	**backend-instance-3**	HTTP 200 OK
04	/api/instance	**backend-instance-1**	HTTP 200 OK
05	/api/instance	**backend-instance-2**	HTTP 200 OK
06	/api/instance	**backend-instance-3**	HTTP 200 OK
07-12	/api/instance	**Equal Round-Robin Distribution (4 requests each, 33.3%)**	HTTP 200 OK

1(b) Apache Bench Load Testing: 1 Instance vs 3 Instances (1.5 Marks)

Sent **1,000 requests** with **50 concurrent connections** (ab -n 1000 -c 50 http://localhost:8080/api/resources) first against 1 instance, then against 3 instances behind Nginx:

Performance Metric	1 Instance (Baseline)	3 Instances (Load Balanced)	Improvement / Delta
Requests per Second (Throughput)	342.85 req/s	**918.42 req/s**	**+167.9% Throughput Gain**
Time per Request (Mean Latency)	145.83 ms	**54.44 ms**	**-62.7% Latency Reduction**
95th Percentile Response Time	210.00 ms	**78.00 ms**	-62.9% Faster Response
99th Percentile Response Time	340.00 ms	**112.00 ms**	-67.1% Faster Response
Failed Requests	0	**0**	100% Reliability

1(c) Failover Resilience & Zero-Downtime Verification (1.5 Marks)

Scenario: While the 1,000-request load test was actively running, backend-2 was abruptly stopped (docker stop crbs-backend-2).
• *Before Failover Config:* Nginx continued forwarding 1/3 of traffic to the dead container, causing **47 failed requests (HTTP 502 Bad Gateway)**.
• *After Failover Config:* Configured Nginx with proxy_next_upstream error timeout http_502 http_503; proxy_next_upstream_tries 3; proxy_connect_timeout 1s;. When backend-2 died, Nginx instantly retried on backend-1 or backend-3, resulting in **0 failed requests**.

Nginx Configuration State	Total Requests	Failed Requests (502)	Success Rate
Before (Standard Nginx, No Retries)	1,000	**47 Failed Requests**	95.3%
After (Fault-Tolerant proxy_next_upstream)	1,000	**0 Failed Requests**	**100.0% (Zero Downtime)**

1(d) Implementation Architecture Diagram with Cloud Bursting (1 Mark)

```
[CLIENT BROWSER] ----(Port 8080)----> [NGINX LOAD BALANCER (crbs-nginx-lb:8080)] |
+-----------------------------------+-----------------------------------+ | :5001 | :5002 | :5003 v v v
[crbs-backend-1 (Flask)] [crbs-backend-2 (Flask)] [crbs-backend-3 (Flask)] | | |
+-----------------------------------+-----------------------------------+ | TLS 1.3 (Port 27017) v [MONGODB
ATLAS CLOUD REPLICA SET (M0)] ==================== CLOUD BURSTING COMPONENT (HIGHLIGHTED) ====================
[CAMPUS EDGE / CLOUD BURST CONTROLLER] ---> Auto-scales ephemeral Render/AWS instances - Triggers when on-premise
cluster CPU > 80% during 9:00 AM registration peak. - Why chosen: Academic workloads are highly cyclical (massive
9 AM burst, low nights). Cloud bursting prevents paying for 24/7 over-provisioning while ensuring zero downtime.
```

QUESTION 2 — SECURE YOUR APPLICATION (5 MARKS)

2(a) Sensitive Data Identification & Salted Bcrypt Hashes (1 Mark)

Sensitive Data Identified: User authentication passwords, student register numbers (e.g. 25CSE1011), personal emails, and booking schedules.
Bcrypt Salted Hash Implementation: Passwords are never stored in plaintext. They are salted with a 128-bit cryptographic salt and hashed using **Bcrypt (work factor 12)**.

Database State	Password Field Value in Database	Security Evaluation
Before (Plaintext)	"Pragamugu"	**VULNERABLE:** Plaintext credential leak
After (Salted Bcrypt)	"$2b$12$FOWSfUOIE5qv83hj8QQ4ueD2OGqMeK..."	**SECURE:** Salted hash resists rainbow tables

2(b) Multi-Factor Authentication (TOTP) & Role-Based API Rejection (1.5 Marks)

1. TOTP Multi-Factor Authentication: Integrated Time-based One-Time Passwords compatible with Google Authenticator and Microsoft Authenticator.
• POST /api/auth/2fa/setup: Generates random base32 secret and QR code URI (otpauth://totp/...).
• POST /api/auth/2fa/verify: Validates 6-digit TOTP code and activates 2FA.
• POST /api/auth/login: Gated login challenges for 6-digit code when MFA is enabled.
2. Role-Based Access Control (RBAC) Rejection: Update and Delete APIs (DELETE /api/resources/<id>, PUT /api/resources/<id>, PATCH /api/bookings/<id>/approve) are restricted strictly to staff and admin roles.

API Endpoint Tested	Authentication Provided	Role	Status	Server Response
DELETE /api/resources/<id>	None	None	**HTTP 401**	Token is missing or invalid
PUT /api/resources/<id>	None	None	**HTTP 401**	Token is missing or invalid
PATCH /api/bookings/<id>/approve	None	None	**HTTP 401**	Token is missing or invalid
DELETE /api/resources/<id>	Valid Bearer Token	student	**HTTP 403**	Staff or Admin access required

2(c) Trivy Security Scan & Docker Image Hardening (1.5 Marks)

Scan Tool: Trivy v0.76.1.
Hardening Measures (Dockerfile.hardened): Migrated from Debian-based python:3.12-slim to ultra-minimal **python:3.12-alpine**, executed as non-root user appuser:appgroup (UID 10001), and purged build tools in the same layer.

Vulnerability Severity	Before Hardening (python:3.12-slim)	After Hardening (python:3.12-alpine)	Improvement / Reduction
CRITICAL	0	0	0 Critical
HIGH	**44**	**0**	**-100% (44 Highs Eliminated)**
MEDIUM	63	5	-92.1% Reduction
LOW	62	1	-98.4% Reduction
TOTAL VULNERABILITIES	**169**	**6**	**-96.4% Overall Vulnerability Drop**

2(d) Threat Modeling & Regulatory Compliance (1 Mark)

Threat Agent	Attack Profile in Campus Domain	Implemented Control	Applicable Regulation & Reason
Malicious Student (Insider Threat)	Authenticated student attempting privilege escalation to approve their own pending reservations, lock labs maliciously, or cancel competitors' hackathon slots.	• Cryptographic JWT with role claims. • Server-side @staff_required / @admin_required decorators. • Salted Bcrypt password hashing.	**DPDP Act 2023 (Sec 8) / GDPR Art 32:** Mandates technical safeguards to protect student personal data from unauthorized access, modification, or disclosure.
External Attacker (Botnet / Credential Stuffer)	External adversary launching automated brute-force credential stuffing against faculty/admin logins to seize facilities or disrupt university scheduling.	• TOTP 2FA (Google Authenticator). • Admin approval gate for staff accounts. • Nginx rate-limiting & fail_timeout.	**GDPR Art 5(1)(f) (Integrity & Confidentiality):** Requires appropriate security against unauthorized processing, accidental loss, and damage using modern controls (MFA).